

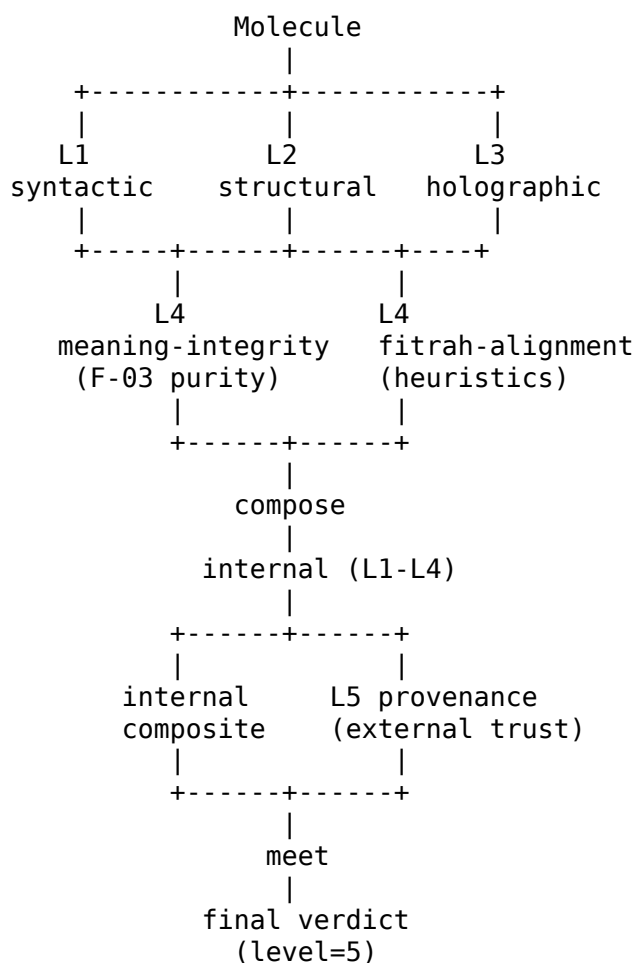
FATIMA-Core Skeleton Schematic - End of Day

Codebase: fatima-core at commit 4eale80 **Date:** 2026-10-07 **Test suite:** 34/34 passing (0.20s) **Canon findings closed today:** F-03, F-04, F-01, F-05 (4 of 8)

1. Module Map

```
fatima-core/  
  fatima/  
    __init__.py  
    cli.py # Entry point (not yet provenance-aware)  
    core/  
      __init__.py  
      atom.py # [GRAFTED F-03] Purity separation  
      bond.py # Frozen dataclass, computed bond_id  
      composite.py # Composite data structures  
      encoding.py # GF(2^8) Shamir encoding  
      molecule.py # [GRAFTED F-03] Purity separation  
    formats/  
      __init__.py  
      document.py # Document-level format handling  
      serialization.py # Serialization/deserialization  
    graft/  
      __init__.py # Graft protocol package  
      anchor.py # Anchor-hash protocol  
      composite.py # Composite graft orchestration  
      independence.py # Independence attestation  
      manifest.py # Graft manifest dataclass  
      report.py # GraftReport dataclass  
      adversarial/  
        __init__.py  
        fixtures.py # Adversarial test fixtures  
    properties/  
      __init__.py  
      authenticity.py # [GRAFTED F-01/F-05] P5 wrapper  
      fitrah.py # P2 Fitrah-Alignment  
      holographic.py # P1 Holographic Redundancy  
      meaning.py # [GRAFTED F-03] P4 Meaning-Integrity  
      tawhidic.py # P3 Tawhidic Unity  
    verification/  
      __init__.py  
      composite.py # Composite verification  
      holographic.py # L3 holographic verification  
      provenance.py # [NEW] L5 provenance verification  
      semantic.py # [GRAFTED F-04/F-01/F-05] L4+L5  
      structural.py # L2 structural verification  
      syntactic.py # L1 syntactic verification  
      verdict.py # Verdict enum, VerificationReport  
  tests/  
    __init__.py  
    test_graft_f03_purity.py # 4 tests -- F-03 purity graft  
    test_graft_f04_f01_f05.py # 12 tests -- coupled F-04/F-01/F-05  
    test_graft_protocol.py # 18 tests -- graft protocol infra
```

2. Verification Pipeline - Evolved Architecture



Key invariant: Without a provenance envelope, L5 is UNKNOWN. The final verdict is the lattice meet of internal and provenance. A document cannot reach VERIFIED without external evidence.

3. Canon Findings - Status Register

ID	Finding	Severity	Status	Graft
F-01	Authenticity is forgeable	CRITICAL	CLOSED	F-04/F-01/F-05
F-02	Syntactic verification is circular	HIGH	OPEN	-
F-03	Meaning-integrity is vacuous	CRITICAL	CLOSED	GRAFT-F03-PURITY-001
F-04	P4 is orphaned from pipeline	HIGH	CLOSED	F-04/F-01/F-05
F-05	No external anchoring	HIGH	CLOSED	F-04/F-01/F-05
F-06	Bond inference is lexical	MEDIUM	OPEN	-
F-07	Encoder is slow	MEDIUM	OPEN	-
F-08	Shard corruption fails by exception	LOW	OPEN	-

4. Grafts Applied Today

GRAFT-F03-PURITY-001 - Semantic Hash Purity

Separated commitment calculation from commitment storage.

- `Atom.semantic_hash_value()` - computes without mutation (pure)
- `Atom.compute_semantic_hash()` - build-time commit operation
- `Molecule.bond_graph_hash_value()` - computes without mutation (pure)
- `Molecule.compute_bond_graph_hash()` - build-time commit operation
- `check_meaning_integrity()` - recomputes and compares only, never writes
- Stale commitment - VIOLATED
- Missing commitment - UNKNOWN

F-04/F-01/F-05 Coupled Graft - Provenance Authenticity

Three Canon findings closed in one operation because separating them would leave an intermediate architecture capable of making misleading L5 claims.

- **F-04 closure:** `check_meaning_integrity()` wired into L4 via `VerificationReport.compose(l4_meaning, l4_fitrah)`
- **F-01 closure:** Self-authentication eliminated. Without a provenance envelope, L5 returns UNKNOWN - never VERIFIED.
- **F-05 closure:** `ProvenanceEnvelope` requires author signature, external time anchor, and optional reviewer attestation. `verify_provenance()` checks commitment match, signature against trust store, anchor presence, and reviewer independence.

5. Provenance Data Model

`ProvenanceEnvelope` (frozen)

```
author_key_id:      str          # signing key identifier
signature:          bytes        # over commitment_hash
commitment_hash:    str          # must match molecule.bond_graph_hash
anchor_provider:    str          # "rfc3161", "opentimestamps", "null"
anchor_proof:       bytes        # provider-specific proof
anchor_timestamp:   str          # ISO-8601
reviewer_key_id:    str|None     # independent reviewer
reviewer_attestation: bytes|None
```

`PublicKeyProvider` (Protocol)

```
get_public_key(key_id: str) -> object|None
```

`NullKeyProvider`

Always returns None -- honest default when no trust store configured.

verify_provenance() checks (4 total):

1. Commitment hash matches expected - VIOLATED on mismatch
2. Author signature verifies against trust store - UNKNOWN if key absent
3. External time anchor present - UNKNOWN if null provider
4. Reviewer independence - VIOLATED on key identity collapse

6. Verdict Algebra

VERIFIED > UNKNOWN > VIOLATED

```

meet (lattice infimum, the & operator):
    VERIFIED & VERIFIED = VERIFIED
    VERIFIED & UNKNOWN  = UNKNOWN
    VERIFIED & VIOLATED = VIOLATED
    UNKNOWN  & UNKNOWN  = UNKNOWN
    UNKNOWN  & VIOLATED = VIOLATED
    VIOLATED & VIOLATED = VIOLATED

compose(report_1, report_2, ...):
    verdict = meet(all verdicts)
    violations = concat(all violations)
    confidence = min(all confidences)

```

7. Test Suite Structure

tests/ -- 34 tests, 3 files

```

test_graft_f03_purity.py (4 tests)
    test_semantic_hash_value_is_pure
    test_compute_semantic_hash_commits
    test_meaning_integrity_verified_after_commit
    test_meaning_integrity_detects_tampering

test_graft_f04_f01_f05.py (12 tests)
    F-04 wiring:
        test_f04_meaning_integrity_wired_into_verify_authenticity
        test_f04_missing_commitments_degrade_full_pipeline
    F-01 self-auth elimination:
        test_f01_no_provenance_yields_unknown_not_verified
        test_f01_check_authenticity_wrapper_also_unknown_without_provenance
        test_f01_is_authentic_false_without_provenance
    F-05 external anchoring:
        test_f05_provenance_with_mismatched_commitment_is_violated
        test_f05_provenance_with_matching_commitment_but_no_key_is_unknown
        test_f05_provenance_null_anchor_is_unknown
        test_f05_self_attestation_detected
        test_f05_no_provenance_envelope_is_unknown
    Integration:
        test_full_pipeline_with_provenance_envelope_unknown_key
        test_full_pipeline_provenance_through_check_authenticity

test_graft_protocol.py (18 tests)
    Graft infrastructure: manifest, independence, anchor, composite,
    adversarial fixtures

```

8. Adversarial Audit Register - v0.2 Prototype (Forward-Looking)

These findings are from the adversarial audit of the v0.2 executable prototype. They are recorded here as the open register for future grafts. None have been applied yet.

ID	Severity	Finding	Proposed Repair
F-V02-01	HIGH	TrustStore provenance is unenforced	R-1: Declare trust store boundary
F-V02-02	HIGH	Attestation has no declared criterion	R-2: Extend SemanticAttestation
F-V02-03	MEDIUM	Nominal independence, not effective	R-3: AnchorVerifier protocol
F-V02-04	MEDIUM	No revocation, cryptoperiod, or re-anchoring	R-4: Crypto-agility fields
F-V02-05	MEDIUM	anchor_ref is opaque/unverified	R-3: AnchorVerifier protocol
F-V02-06	MEDIUM	Purity assertion narrower than the claim	R-4: Widen purity tuple
F-V02-07	MEDIUM	L2 largely redundant with construction-time checks	R-7: Strengthen or rename L2
F-V02-08	LOW	Share accepts unvalidated inputs	R-5: Share.__post_init__
F-V02-09	LOW	bond_type is a free string with no vocabulary	R-6: Bond-type schema registry
F-V02-10	INFO	No structured evidence in attestation	R-2 (extended)
F-V02-11	INFO	Migration module is a stub	R-8: Rename stub

9. Graft Protocol Infrastructure

The codebase now includes a reusable graft protocol under `fatima/graft/`:

- **GraftManifest** - declares finding, hypothesis, affected files, before/after hashes, and the regression contract
- **GraftReport** - records verdict (UNKNOWN pending independent review), baseline reproduction, and candidate results
- **Independence attestation** - verifies graft author != test author (nominal, with cross-references to the effective-independence problem)
- **Anchor hash** - SHA-256 ledger of all modified files
- **Composite orchestration** - multi-graft sequencing
- **Adversarial fixtures** - reusable test fixtures for graft testing

All graft reports carry verdict UNKNOWN until an independent seat reviews the work. The graft protocol itself is subject to the same self-graded-homework constraint it measures.

10. What Remains

Canon grafts still open (4 of 8):

- F-02: Syntactic verification is circular - L1 checks syntactic validity against its own construction rules. A forged graph that satisfies the dataclass invariants passes L1.
- F-06: Bond inference is lexical - bonds are currently declared by the document producer, not inferred from content. The system verifies declared bonds, not meaning.

- F-07: Encoder is slow - $GF(2^8)$ Shamir encoding is polynomial. For large documents, this is a practical bottleneck.
- F-08: Shard corruption fails by exception - a corrupted shard raises an unhandled exception rather than producing VIOLATED.

Adversarial audit repairs (11 findings, 0 applied):

The v0.2 prototype audit (F-V02-01 through F-V02-11) produced 11 findings with 9 proposed repairs (R-1 through R-9). These are separate from the Canon grafts and target the provenance and attestation subsystems.

Honest boundary: The architecture after today's grafts can detect meaning-integrity violations and can verify external provenance when an envelope is supplied. It cannot enforce that the trust store is independent (authority regress), cannot enforce effective reviewer independence (only nominal key separation), and cannot verify the content of a semantic attestation (only that one was signed). These are declared limits, not hidden ones.